

Capitolo 7

File System

7.1 Introduzione al File System

I **file**, le **directory** e il **file system** sono astrazioni offerte dal sistema operativo. La gestione di **grandi quantità d'informazioni**, in maniera persistente e condivisa tra processi, è l'obiettivo che ci poniamo durante la progettazione di un sistema operativo.

7.1.1 Il File system

Il termine **file system** viene spesso utilizzato per indicare due cose:

- L'interezza dei file in memoria (Cancellare l'intero file system → Cancellare tutti i file in memoria).
- L'astrazione di gestione dei file (Il file system di UNIX è diverso da quello di Windows).

Quando useremo il termine file system, ci riferiremo sempre all'astrazione di gestione dei file. Ogni **file system** si differenzia per i propri **dettagli di gestione** e di **implementazione**. Tra questi dettagli di gestione, individuiamo:

- **Nomenclatura.**
Dimensione e formattazione dei nomi (MS-DOS: 7 caratteri + 3 per l'estensione), case sensitivity (UNIX-based OS - case sensitive, Windows - case insensitive).
- **Tipi di file.**
Tra questi, nei sistemi operativi UNIX, abbiamo i **device file**. Rappresentano un dispositivo che appare nel file system, e tramite cui è possibile effettuare operazioni di input/output. Una scheda audio, ad esempio, apparirà come un file nel file system.
- **Tipi di accesso.**
Sequenziale o diretto. Nel primo, il file pointer viene portato avanti ad ogni operazione di lettura e scrittura. Nel secondo, il file pointer è completamente svincolato. Al giorno d'oggi l'accesso diretto è lo standard, e permette di emulare anche un accesso sequenziale.
- **Metadati e attributi.**
Tra cui nome, time-stamp (data creazione - ultima modifica), permessi di modifica, autore del file e maschere di permessi, tramite cui è possibile stabilire accesso protetto ad un file. Ogni utente potrebbe avere il proprio spazio nel file system.
- **Operazioni supportate sui file.** Consentire a due processi di leggere e scrivere sullo stesso file, ci porta alle stesse condizioni delle race condition. Il sistema operativo gestisce l'accesso condiviso ai file tramite dei **lock**. I lock possono essere:
 - **Condivisi o Esclusivi.**
Sono due tipologie di lock ben distinte. I lock **condivisi** consentono a più processi di accedere allo stesso file, e vengono utilizzati quando più processi vogliono **leggere** dallo stesso file. I lock esclusivi garantiscono accesso esclusivo ad un processo su quel file. Questo lock preclude l'accesso a qualsiasi altro processo, e viene usato solitamente per operazioni di **scrittura**.
 - **Mandatory o Advisory.**
Con la strategia **mandatory**, i processi sono **obbligati** a rispettare i lock tramite l'intervento diretto del sistema operativo, che blocca il processo che ha intenzione di accedere al file lockato.

Con la strategia **advisory** il sistema operativo informa il processo se è riuscito o meno ad ottenere un lock sul file che ha intenzione di usare. Sconsiglia al processo di operare sul file trovato bloccato, ma non gli vieta di farlo.

- **Strutture dati** per la gestione dei file. Esistono tabelle globali per visualizzare i file in uso.

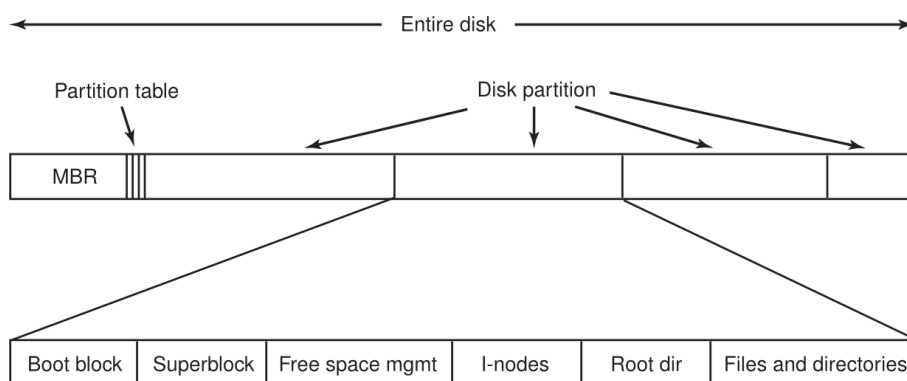
7.2 Struttura di un File system

Il file system risiede sopra una partizione di memoria. Una partizione può ospitare più file system. La gestione delle partizioni non concerne quindi il file system. Il partizionamento può essere gestito in due modi principali.

7.2.1 Master Boot Record

È un metodo storico. Il master boot record contiene quattro voci prestabilite, che tracciano l'inizio e la fine delle partizioni di memoria (esiste quindi un cap limite di massimo 4 partizioni di memoria). Il disco ha quindi in memoria:

- Al più quattro partizioni.
- La tabella delle partizioni.
- Il boot record. È il primo blocco della memoria, i primi 512 byte, contenente codice di avvio della macchina. Carica il sistema operativo presente nella rispettiva partizione.



All'interno di ogni partizione, individuiamo:

- **Boot block.**
Una volta sfruttato per parte del codice di avvio di ciascuno dei sistemi operativi delle partizioni. Il suo utilizzo, che ne spiegava il nome, è ormai deprecato. Rimane ormai inutilizzato, vuoto.
- **Superblock.**
Contiene dei metadati relativi alla partizione: specifica il tipo di file system, la dimensione dei cluster. Il resto della partizione sarà gestita in cluster.
- **Free space management.**
- **I-nodes.**
- **Root directory.**
La radice della directory del file system.
- File e directories.

7.2.2 GUID Partition Table

È un layout moderno alternativo, preferito al MBR. Una partizione prestabilita, la partizione EFI, ha un compito dedicato per l'avvio del sistema. La partizione EFI è formattata in FAT e all'interno del file system esistono vari file di configurazione e gli eseguibili EFI da lanciare in un ambiente minimale. Questi binari si occuperanno dell'avvio dei sistemi operativi.

7.3 Implementazione dei file

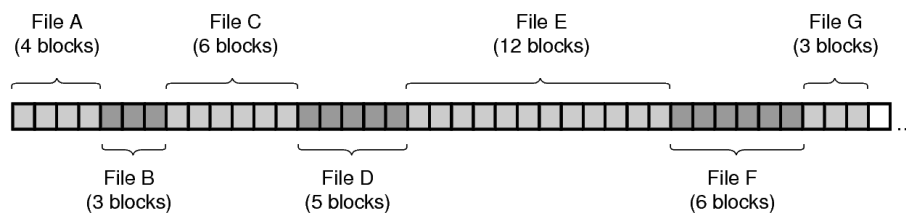
Definire le strategie relative all'implementazione dei file, è fondamentale per capire il funzionamento del file system.

7.3.1 Strategie principali

Ogni oggetto file ha una dimensione specifica in blocchi. Lo spazio richiesto in blocchi è il risultato di un arrotondamento: a ogni file, coinciderà uno spreco (frammentazione interna) causata dalla differenza tra la dimensione di un blocco e la taglia del file.

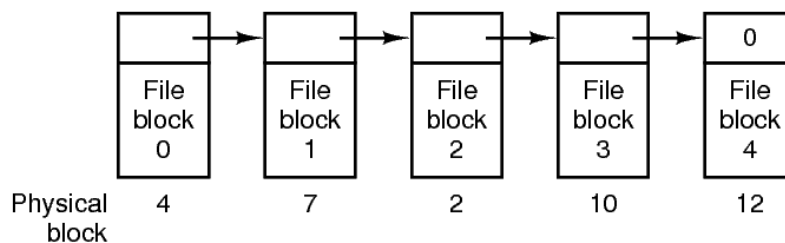
- **Allocazione contigua.**

I blocchi relativi ad un file devono essere contigui. Permette l'accesso diretto ad un blocco in tempo $O(1)$: una sola lettura, sapendo già indirizzo e offset. Sfortunatamente, l'allocazione contigua è ingestibile in situazioni di crescita dinamica dei file. Il vincolo di contiguità è troppo stretto: cosa facciamo se il prossimo blocco è occupato? Spostiamo tutti i blocchi del file da qualche altra parte? Troppo oneroso. Il metodo è utilizzabile solo nei file system di sola lettura.



- **Allocazione con liste collegate.**

Detta anche **allocazione concatenata**. Ogni blocco del file conterrà un indirizzo (di un numero di byte commisurato alla memoria della partizione), che farà riferimento al prossimo blocco del file. Questa strategia presenta un problema: con n blocchi dedicati a un file, la ricerca di un qualsiasi blocco richiederà $O(n)$ accessi in memoria.



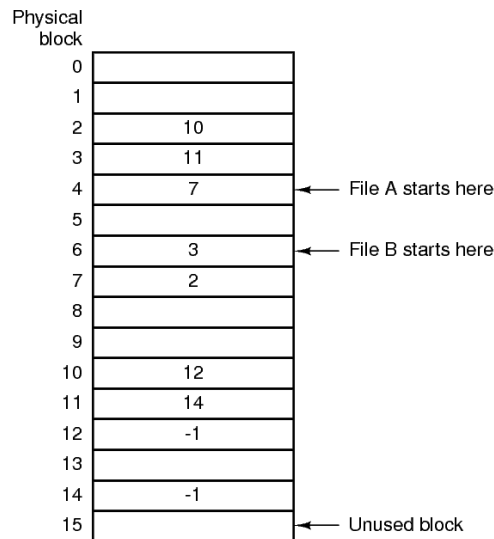
- **FAT - File Allocation Table.**

I blocchi non devono essere contigui. In più, in memoria è presente una tabella dei file allocati: ha un record per ogni blocco. Ecco come funziona:

- La FAT ha un record per ogni blocco della partizione.
- Nel record del blocco i , è contenuto l'indice del prossimo blocco, rispetto al file di i .
- Se è presente l'indice -1 , il blocco sarà l'ultimo del file di appartenenza.
- I **record vuoti** coincidono a **blocchi inutilizzati**¹.

Il FAT è di fondamentale importanza per il file system: il file system non esiste senza questa tabella! Notiamo inoltre che riprende gli stessi presupposti dell'allocazione con liste collegate (non contiguità, riferimenti tra blocchi). Per trovare un blocco qualsiasi di un file con n blocchi, sono richiesti $O(n)$ accessi all'interno della tabella. Supponendo però che l'intera FAT sia caricata in RAM, la ricerca all'interno della tabella richiederà $O(n)$ passaggi meno computazionalmente onerosi, rispetto agli $O(n)$ accessi in memoria delle liste collegate.

¹Scopriremo poi che altri file system, come gli i-node, implementano strutture per tracciare i blocchi inutilizzati. Per i file system FAT non è necessario! Le informazioni necessarie sono già presenti nella FAT.

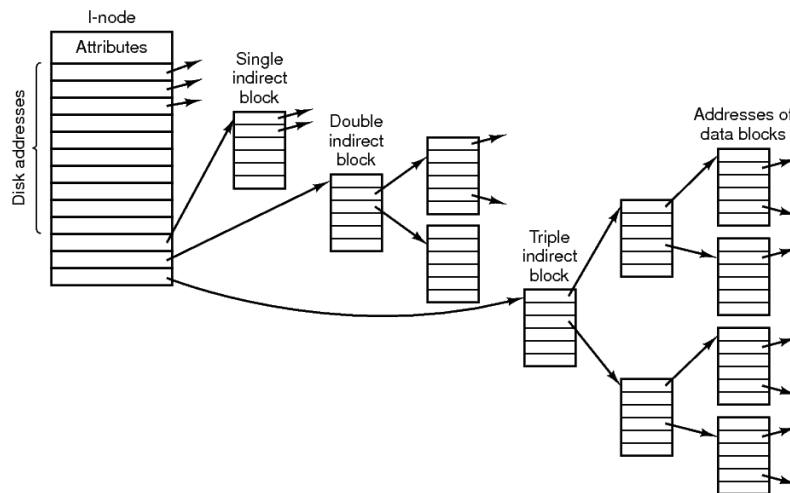


- **Allocazione con i-node**, index-node, nodi indice.

I metadati di un file vengono scritti all'interno della tabella i-node (ad eccezione del nome, che è contenuto nella directory del file). Tra questi metadati abbiamo informazioni relative allo UID e il GUID dell'owner, la dimensione del file e vari timestamps. Tra questi, alcuni sono detti **data block pointers**: questi, sono puntatori a blocchi di memoria.

Questi indirizzi, possono puntare a

- **Un blocco diretto**: è un blocco effettivo del file.
- **Un blocco indiretto singolo**: questo, punta a n blocchi del disco.
- **Un blocco indiretto doppio**: questo, punta a n blocchi indiretti singoli.
- **Un blocco indiretto triplo**: questo, punta a n blocchi indiretti doppi.



Nel file system Unix un i-node contiene quindici puntatori:

- 10 puntatori a blocco diretto. Uno a a uno.
- Un puntatore a blocco indiretto singolo. Per arrivare ad un blocco, sono richiesti al più due accessi in memoria. Uno a n .
- Un puntatore a blocco indiretto doppio. Per arrivare ad un blocco, sono richiesti al più tre accessi in memoria. Uno a n^2 .
- Un puntatore a blocco indiretto triplo. Per leggere un blocco, sono richiesti quattro accessi in memoria. Uno a n^3 .

Questa struttura fornisce un upper bound alla ricerca di un blocco di un qualsiasi file, a partire da qualsiasi i-node. Inoltre, a differenza della FAT, non c'è l'esigenza di portare in memoria l'intera tabella: bensì, i blocchi diretti, indiretti singoli, doppi e tripli, verranno portati in memoria RAM solo on-demand. I sistemi moderni prediligono gli i-node, anche se le chiavette usano la FAT32.

7.4 Implementazione delle directory

7.4.1 Dove vengono inseriti i metadati e attributi?

- In un file system FAT, i metadati sono contenuti all'interno della directory del file.
- In un file system con i-node, i metadati sono contenuti all'interno della tabella degli i-node, **tranne il nome!** Il nome sta nella directory.

Tutti gli attributi hanno una lunghezza fissata, **ad eccezione dei nomi**. Nei sistemi operativi moderni, dove il nome può raggiungere centinaia di caratteri e codifiche UNICODE più dispendiose, è fondamentale conservare in maniera opportuna il nome del file.

Come gestire in maniera opportuna i nomi

Le strategie in questione variano relativamente poco tra i file system FAT e con i-node.

- **Strategia 1 - lunghezza variabile.**

Il primo campo del file stabilisce **la lunghezza in byte dell'entry**, (o del nome, negli i-node). L'unico valore di dimensione variabile è il nome: ponendo il nome come ultimo attributo del file, diventa possibile stabilire quando inizia e quando finisce il nome. A fine nome, è presente un **carattere di terminazione**.

- **Strategia 2 - heap, suddivisione tra parte fissata e variabile.**

Le parti di dimensione fissa dei record, vengono unificate in una parte di memoria. In questo modo, si evita totalmente la frammentazione esterna. Tra gli attributi, si ha un puntatore al nome del file. Le parti variabili (i nomi) si trovano in una sezione a parte, chiamata heap. Un'accozzaglia di nomi, separati da carattere di terminazione. Separare parte variabile e fissa, permette di facilitarne la gestione.

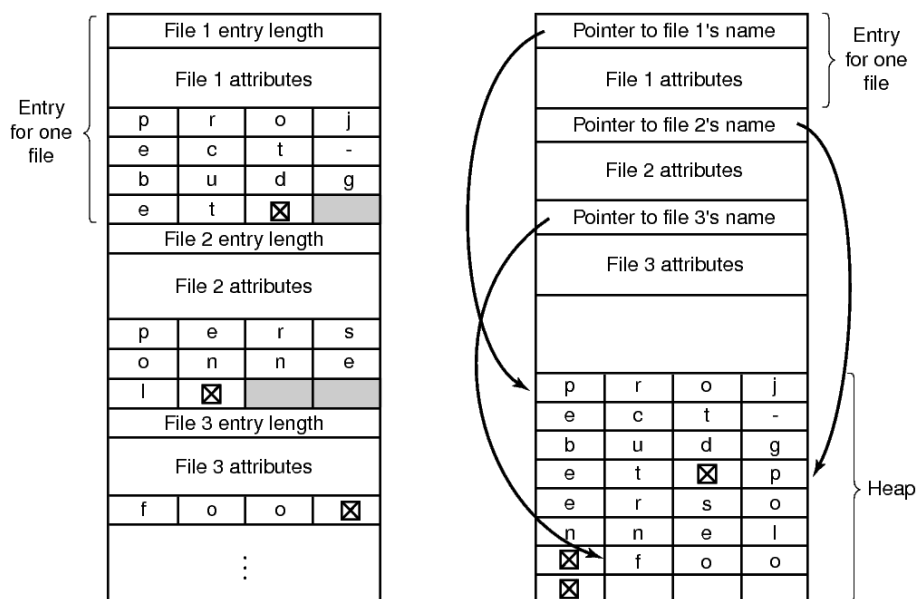


Immagine 7.1: Strategia 1 e 2 a confronto.

7.5 Condivisione di un File System tra utenti

Immaginiamo di voler condividere un file tra due o più utenti.

7.5.1 Alcune idee implementative

- **Duplicando le FAT.**

L'idea potrebbe essere quella di duplicare le liste con i riferimenti ai blocchi ottenendo liste distinte. Come possiamo mantenere coerenti le modifiche al contenuto e ai metadati?

Con un'append, si aggiorna la dimensione in byte: l'aggiornamento dei metadati diventa un problema.

- **Usando i-node, gli hard link.**

All'interno di un i-node, inseriamo un contatore che tiene traccia degli utenti che hanno accesso al file. Possiamo mettere riferimenti diversi allo stesso file. Non ci sarà l'esigenza di aggiornare i metadati, in quanto questi sono situati proprio dentro l'i-node. Il nome può infatti essere diverso tra i vari utenti!

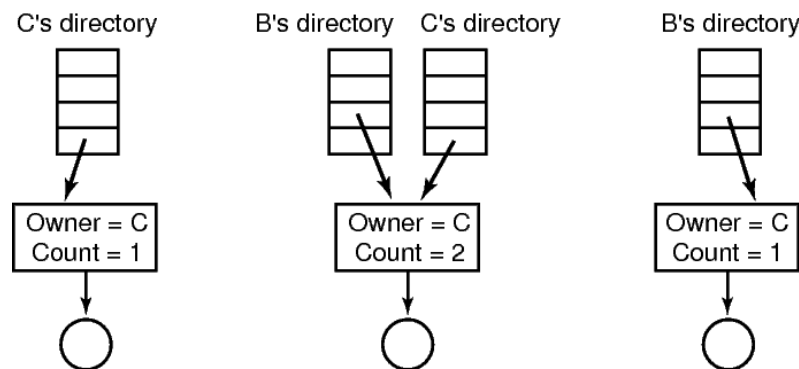


Immagine 7.2: Anomalia con accounting. È impossibile risalire all'owner originale del file.

- **Soft link**, o link simbolici.

Viene creato un i-node per un nuovo oggetto. Questo oggetto conterrà il path al file di cui vogliamo creare il soft-link.

Soft e Hard link a confronto

Nei sistemi UNIX sono supportati entrambi i tipi di link.

- **Creare directory condivise.**

È (teoricamente) possibile con entrambi i tipi di link.

- **Capire qual è il riferimento originale.**

Con la tecnica degli hard link, il riferimento originale è indistinguibile dall'hard link creato successivamente. Ciò potrebbe modificare la struttura ad albero del file system, creando un grafo ciclico, rischiando di creare loop infiniti. Per questo motivo, UNIX vieta gli hard link a directory.

- **Cross file-system links.**

Impossibili tramite gli hard link. UNIX gestisce più file systems agganciandoli all'albero del file system originale. In questo modo, è sempre possibile trovare un cammino dalla root a qualsiasi file di qualsiasi file system. Windows usa un approccio a dischi e file system separati.

Perché non è possibile avere collegamenti tra file systems tramite hard link? L'enumerazione degli i-node è relativa ad ogni file system. È ad esempio impossibile creare un hard link tra la directory `/home` e la directory `/tmp`².

²In Unix, the `/tmp` directory will often be a separate disk partition. In systems with magnetic hard disk drives, performance (overall system IOPS) will increase if disk-head movements from regular disk I/O are separated from the access to the temporary directory.

7.6 Gestione dei blocchi liberi

Anche nel caso dei file system, è fondamentale il tracciamento dei blocchi liberi.

7.6.1 Alcune strategie

- **Usare una bitmap.**

Relativamente piccola, ha una dimensione inversamente proporzionale alla dimensione dei blocchi e direttamente proporzionale al numero di blocchi. È conservata nel disco, ma può essere portata in RAM a fine di velocizzarne l'utilizzo.

- **Usare liste concatenate.**

Un'enorme lista concatenata di **blocchi contenenti indirizzi a blocchi liberi**. Richiede più spazio, ma la struttura è unificata, sfrutta blocchi liberi in memoria. È interessante osservare come la lista dei blocchi liberi si accorcia quando vengono utilizzati blocchi. Un blocco viene sottratto? La lista viene accorciata, liberando spazio sul disco.

Un'ulteriore ottimizzazione potrebbe essere inserire un contatore ad un dato indirizzo, per indicare il numero di blocchi contigui successivi.

La FAT che approccio usa?

La FAT ha già un record per ogni blocco. Dando un valore specifico al numero del blocco non allocato (magari un -2 o uno 0), si possono marcare facilmente i blocchi liberi. Entrambi i meccanismi sarebbero ridondanti.

E i-node?

Usano la strategia bitmap. Questa è inserita in uno spazio dedicato (presente in ogni partizione) chiamato **free space management**. Contiene una bitmap per i blocchi liberi, e una bitmap per gli i-node liberi.

7.7 Controlli di consistenza

Crash di sistema e problemi con l'hardware possono causare inconsistenze nel file system. Lavorare su un file system con delle incoerenze, potrebbe creare problematiche di vario tipo: scrittura consentita su blocchi che sono in realtà allocati per altri file. Parleremo quindi di strategie che permettono di individuare delle incongruenze.

7.7.1 Flag

Quando un file system viene montato, un flag relativo a questo file system viene settato a 1. Quando viene smontato correttamente (e in fase di shutdown) viene settato a 0. Un malfunzionamento lascerà il flag a 1, e quando verrà rimontato, alcune utility avvieranno dei controlli di consistenza.

7.7.2 Controlli di consistenza

Tra i principali controlli di consistenza, abbiamo quelli relativi all'occupazione dei blocchi, e quelli relativi al numero di riferimenti agli i-node.

Sull'occupazione dei blocchi

Si confronta il contenuto della bitmap con l'occupazione effettiva dei blocchi.

Block number															
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	1	0	1	0	1	1	1	1	0	0	1	1	1	0	0
Blocks in use															
0	0	1	0	1	0	0	0	0	1	1	0	0	0	1	1
Free blocks															
(a)															
Block number															
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	1	0	1	0	1	1	1	1	0	0	1	1	1	0	0
Blocks in use															
0	0	0	0	1	0	0	0	0	1	1	0	0	0	1	1
Free blocks															
(b)															
Block number															
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	1	0	1	0	1	1	1	1	0	0	1	1	1	0	0
Blocks in use															
0	0	1	0	2	0	0	0	0	1	1	0	0	0	1	1
Free blocks															
(c)															
Block number															
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	1	0	1	0	2	1	1	1	0	0	1	1	1	0	0
Blocks in use															
0	0	1	0	1	0	0	0	0	1	1	0	0	0	1	1
Free blocks															
(d)															

1. Dati consistenti.
2. Blocco 2 sia libero che occupato.
- 3.
4. Blocco 5 usato da più file. *Qui, ci scappa il morto!*

Sul numero di riferimenti agli i-node

Dentro ogni i-node è presente un campo contenente il numero di riferimenti ad esso. Il controllo di consistenza scorrerà l'intero albero del file system tenendo conto dei riferimenti di ogni i-node.

- Se il numero di volte in cui un i-node è stato individuato nel file system è zero, il contatore deve essere a zero, e quindi l'i-node non è ancora stato allocato.
- Altrimenti, il numero di riferimenti deve coincidere con quello contato.

7.7.3 Journaling

L'idea è la seguente: un file system qualsiasi può essere corredato con un journal³, un log. Il log tiene conto di tutte le operazioni sui meta-dati (e strutture dati, quali i-node, bitmap e così via), tiene conto delle **macro operazioni** su tutti i file. Il journal sta sul disco, e il sistema operativo verifica che sia aggiornato correttamente. Quando una sequenza di istruzioni è stata conclusa con successo, questa viene totalmente cancellata. Se la sequenza di istruzioni non è stata conclusa, e il flag relativo alle anomalie è stato trovato a 1, tutti i passi nel log vengono riefettuati.

Il rischio è che alcune operazioni vengano effettuate più volte (una pre e una post crash). Per questo motivo, le operazioni devono essere idempotenti e non creare problemi se reiterati.

L'obiettivo del journaling è creare un log delle operazioni sui meta-dati, per preservare lo stato dei file system nel caso di crash.

- Un esempio di operazione non idempotente: aggiungi questi blocchi alla lista dei blocchi liberi.
- Un esempio di operazione idempotente: aggiungi questi blocchi alla lista dei blocchi liberi, **se non sono già presenti**.

Non tutte le operazioni sui meta-dati sono inserite nel journal. Memorizzare cambiamenti relativi al contenuto sarebbe molto utile e sicuro, ma anche tanto tanto dispendioso. Garantire l'integrità dei metadati (e quindi delle directory e della struttura del file system) è più importante per non propagare i danni all'intera struttura.

I file system moderni

In una situazione di crash, i file system moderni, limitano i controlli di integrità ai blocchi citati all'interno del journal. Questo velocizza di molto il ripristino della coerenza. È inutile fare un controllo a tappeto.

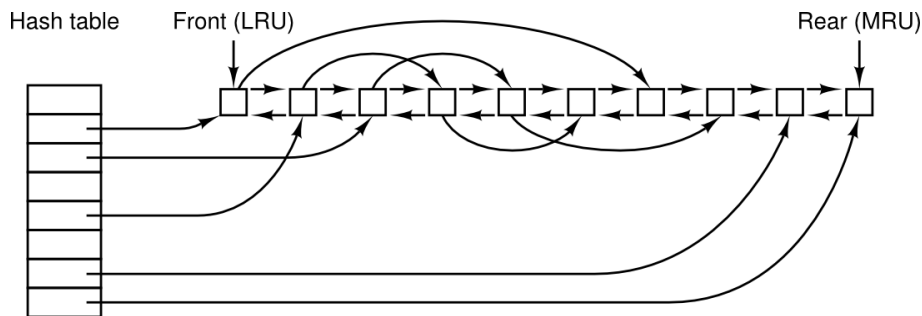
³È una struttura dati che può essere mantenuta nella stessa o in un'altra partizione.

7.8 Cache del disco

Per migliorare le prestazioni dei dischi si usa una **cache del disco**. Fa da ponte tra una memoria veloce ma di piccole dimensioni, e una memoria lenta ma di grandi dimensioni. La cache del disco contiene i dati che sono stati letti di recente e, in alcuni casi, aree di dati adiacenti a cui è probabile che si acceda in seguito. È una **soluzione totalmente software**. Un'idea sarebbe quella di utilizzare la stessa strategia **LRU**, mantenendo un timestamp relativo all'ultimo utilizzo di un blocco.

7.8.1 Struttura basata su tabelle hash.

Con riferimenti a tutti i blocchi presenti nella coda, e in cui individuiamo blocchi **least recently used** (**LRU**) e **most recently used** (**MRU**). La tabella hash consente accessi a tempo costante.



L'implementazione vede in realtà due liste: una per mantenere l'ordine degli elementi rispetto all'ultimo utilizzo, una per accedere agli elementi dell'hash table.

7.8.2 Free-behind e Read-ahead

Immaginiamo di star gestendo un file in lettura o scrittura sequenziale (in generale, un accesso sequenziale, non casuale): ciò che faremo, sarà leggere blocco per blocco il file.

- **Free-behind.**

Effettuata una scrittura sequenziale, si liberano i blocchi precedentemente scritti: raramente immaginiamo di dover leggere gli stessi blocchi.

- **Read-ahead.**

Si ottimizza il processo ad accesso sequenziale, leggendo non solo i blocchi richiesti, ma anche quelli successivi.

Con enormi vantaggi sui dischi elettromeccanici.

Altro da sapere

La cache del disco può essere inibita.

7.9 Che file system usano i nostri sistemi operativi?

7.9.1 Windows

- **exFAT**.
Sui dischi rimovibili
- **NTFS**.
Sui dischi fissi. Supporta **journaling**, **cifratura**, **compressione** (trasparente, prima dell'effettiva scrittura), copia shadow⁴ e dischi multipli (tecniche RAID gestite dal sistema operativo). È un file system moderno e molto complesso.
- **ReFS**, attualmente in sviluppo.
Seguendo la falsa riga di BTRFS.

7.9.2 Linux

- **ext-4**.
Journaling, allocazione efficiente.
- **BTRFS**.
È un file system molto recente. Piuttosto stabile, offre journaling, compressione (trasparente), dischi multipli (con tecniche RAID), checksum dei dati e dei metadati⁵, volumi e clonazione (CoW).

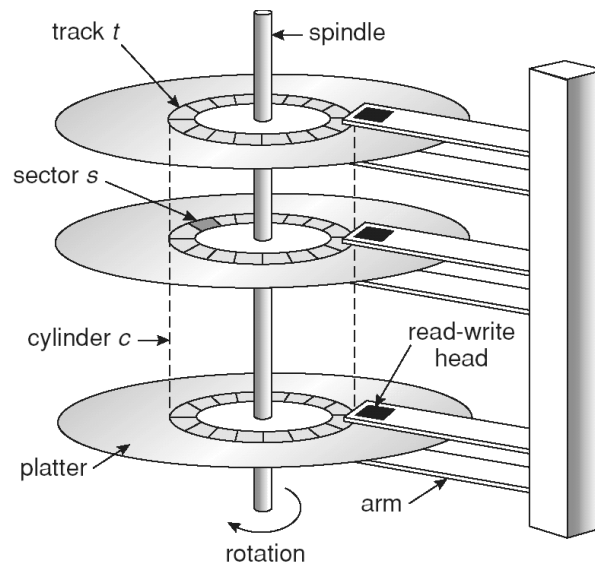
7.9.3 MacOS

HFS+ recentemente soppiantato da APFS.

7.10 Scheduling del disco

Gestione della coda delle richieste pendenti di I/O su disco. Si può mandare al controller del disco solo una richiesta alla volta. Il nostro obiettivo è massimizzare il numero di richieste in unità di tempo (**throughput**), **minimizzando il tempo medio d'accesso**.

Ogni richiesta ha un costo: questo dipende da varie circostanze. Per questo motivo, si possono stabilire delle politiche di selezione per la prossima richiesta da effettuare.



- **FIFO**.
Tecnica banale, non premia particolarmente.

⁴Copy-on-write sui file. Permette di risparmiare spazio in memoria quando si hanno due copie dello stesso file: fino a quando uno dei due non verrà modificato, la copia sarà solo un riferimento agli stessi blocchi di memoria. I file comuni verranno separati nel momento di una modifica.

⁵per garantire congruenza tra i dati e metadati prima di lettura e scrittura, per evitare di leggere dati danneggiati. Un checksum può tornare falsi negativi

- **Tempo di posizionamento più corto.**
O *Seek-time*, metrica che studieremo.
- **Latenza di rotazione.**

Il problema di cui parleremo, è chiamato problema dello **scheduling del movimento della testina su disco**.

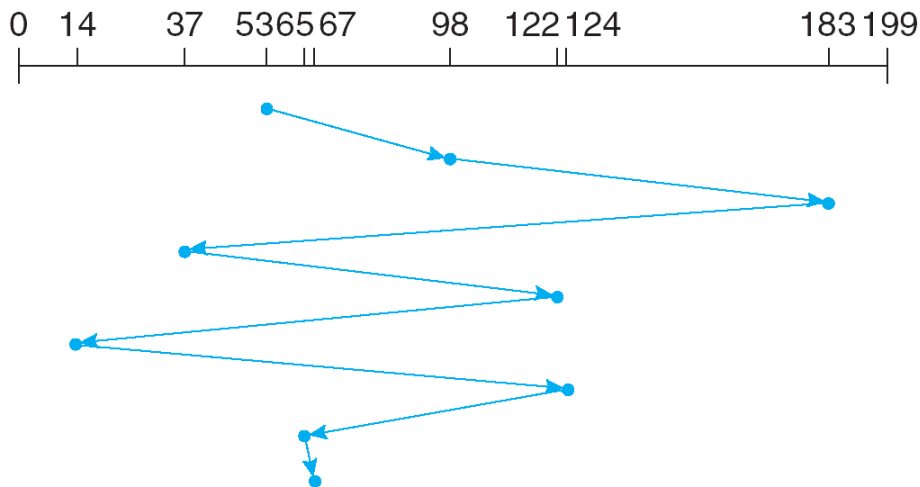
7.10.1 Ottimizzare il Seek-time

La lista delle richieste sarà, partendo dal cilindro 53:

98, 183, 37, 122, 14, 124, 65, 67

1 - Strategia FIFO

First-Come-First-Served.

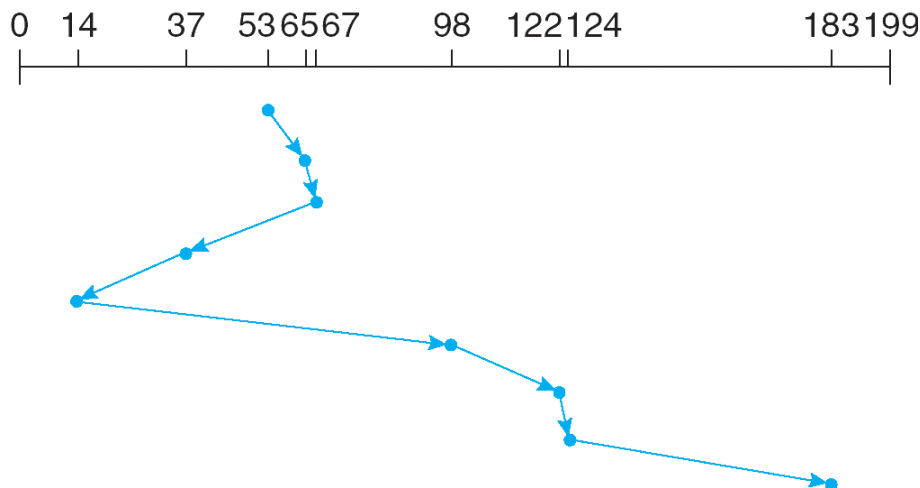


$$(98 - 53) + (183 - 98) + (183 - 37) + (122 - 37) + (122 - 14) + (124 - 14) + (124 - 65) + (67 - 65) = 640$$

Vengono percorse, in totale, 640 tracce. È un algoritmo **semplice** (letteralmente nulla da modificare), **equo**, **ma non efficiente**.

2 - SSTF Shortest Seek Time First

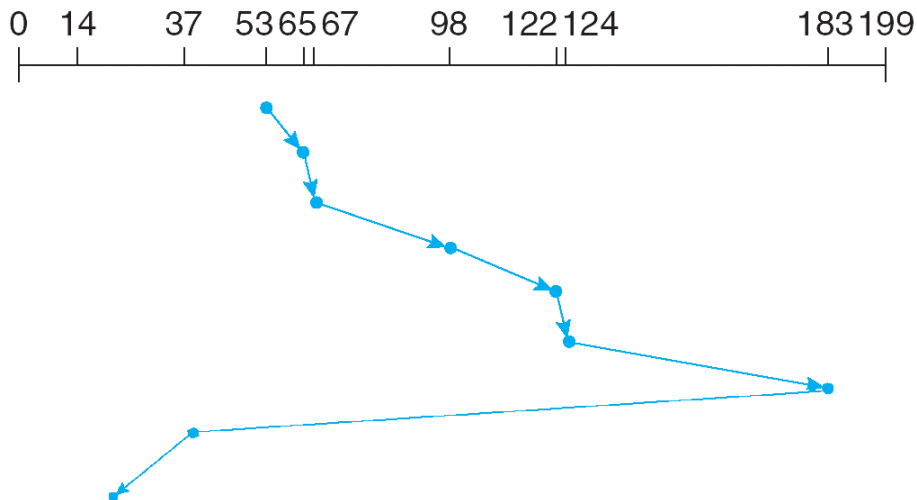
Si minimizza il tempo di ricerca su disco, di ogni richiesta.



236 tracce. Si ottengono ottime prestazioni, ma non è equo (si rischia starvation, soprattutto di fronte ad un alto numero di richieste in arrivo)

3 - Algoritmo dell'ascensore (o scansione, look)

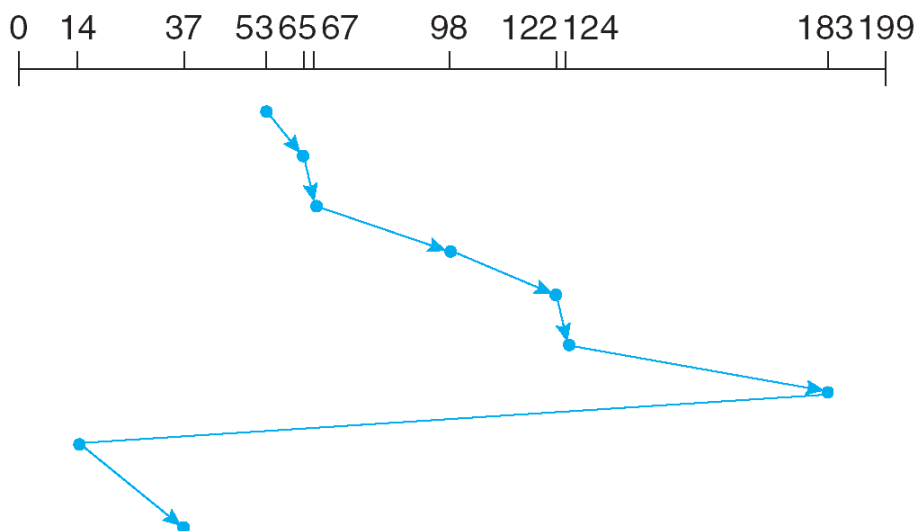
Mantiene un verso fino all'ultima richiesta in tale direzione. Le richieste sono soddisfatte o da sinistra verso destra, o da destra verso sinistra, fino a quando non ci sono richieste da poter soddisfare nel verso stabilito.



299 tracce, più di SSTF. Garantisce un'upper bound ai tempi di attesa per ogni richiesta (in questo caso, 199 tracce), non rischia starvation, e scansiona in maniera uniforme. Il nome deriva dal fatto che, lo stesso algoritmo, viene usato dagli ascensori.

4 - Scheduling per scansione circolare

Considera le posizioni come **collegate in modo circolare**. Arrivato a fine disco (ovvero all'ultima richiesta soddisfacibile dal verso corrente), la testina si sposta alla richiesta più lontana dal verso opposto, ignorando qualsiasi richiesta pendente. È uno spostamento molto veloce.



Con questa sorta di **effetto pac-man**, si abbassa il tempo di attesa medio, in attesa di tante richieste. Il ritorno indietro è molto veloce, minimizzando il tempo di attesa nei casi peggiori.

- In situazioni ad alto carico, il sistema circolare è il migliore.
- A basso-medio carico, si preferisce SSTF o la scansione standard.

7.10.2 Esempio di esercizio

Supponiamo di avere un disco con 200 tracce (numerate da 0 a 199) la cui velocità di seek è di 1 traccia per *ms*. All'istante $t = 0$ il sistema operativo sta servendo una richiesta sulla traccia 100 e in coda ci sono già le seguenti richieste per le tracce (50, 115, 180). Successivamente arrivano altre richieste all'istante $t = 70$ per la traccia 150 e all'istante $t = 130$ per la traccia 90. Si calcoli il tempo di ricerca complessivo (in *ms*) per servire tutte le richieste secondo la politica LOOK (scansione), Iniziando in ordine ascendente (dalla traccia 0 verso la traccia 199) e trascurando la latenza rotazionale e il tempo di trasferimento. Indicare anche la sequenza di scheduling considerata.

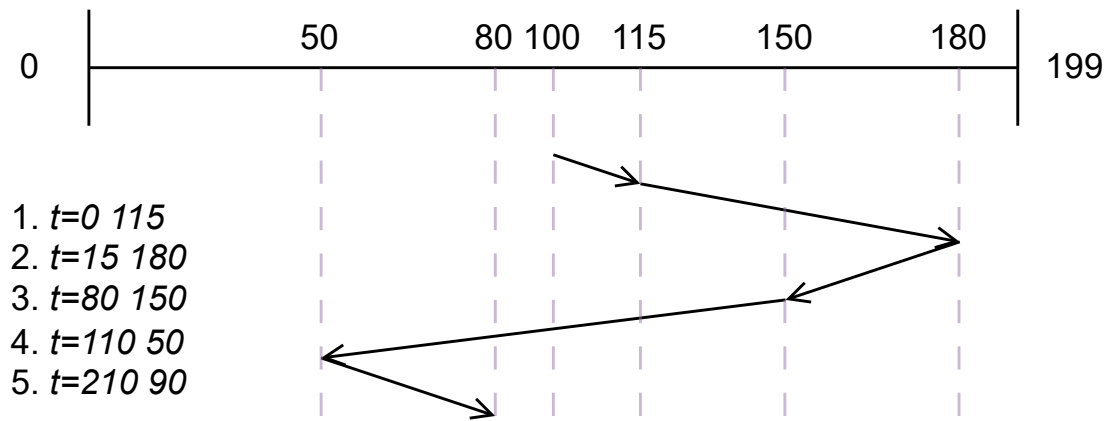


Immagine 7.3: Esercizio svolto in classe

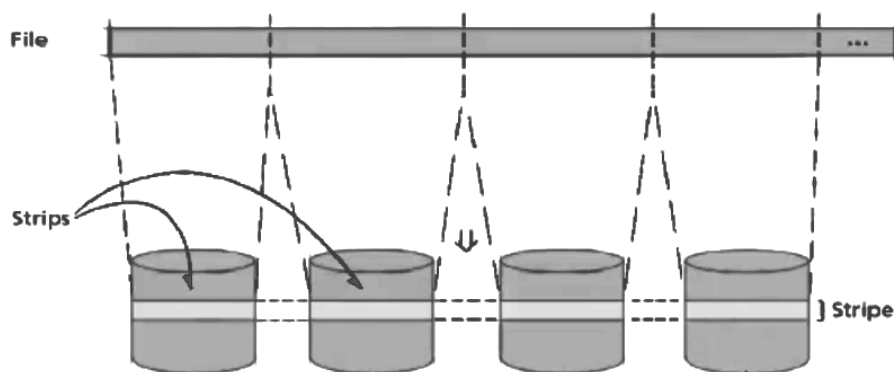
7.11 Parallelismo dei dischi (Sistemi RAID)

I sistemi RAID (*Redundant Array of Independent Disks* - Array Ridondante di dischi indipendenti) offrono una gestione di un'unica disco logico (virtuale) su più dischi fisici. fittizio. Che benefici otteniamo?

Le richieste verso il disco sono pilotate verso il disco virtuale, ma le operazioni di lettura e scrittura verranno effettuate sui dischi effettivi, ovvero tutti i dischi fisici coinvolti. **Permette di sfruttare più dischi indipendenti in maniera parallela**, con fine ultimo quello di **aumentare le prestazioni**, e di migliorare la **resilienza ai guasti**, tramite meccanismi di ricostruzione delle informazioni perse.

7.11.1 Striping

I dati relativi ad un'unità logica (un file, un volume) vengono distribuiti su più dischi: questa procedura è detta **striping**. Gli **stripe** sono multipli del blocco, di medio-grandi dimensioni. Le stripe sono distribuite sui vari dischi in maniera casuale.



I sistemi operativi attuali, preferiscono gestire lo striping tramite software. Si ottengono prestazioni simili tramite hardware (da parte del controller dei dischi).

Rischi relativi all'uso di più dischi - maggiore sensibilità?

Usare più dischi fisici per distribuire un volume logico, significa rendere i dati molto sensibili ai guasti di uno dei dischi coinvolti qualsiasi: i sistemi raid aggiungono ridondanza per ottenere migliore affidabilità, ed è possibile contemplare dischi spare con meccanismi di sostituzione automatica.

Aumento di prestazioni

Le richieste al disco virtuale, sono distribuite parallelamente ai dischi effettivi sottostanti. Questi opereranno in parallelo. Supponendo dischi uguali a pari prestazioni, e con porzioni di file uguali, otterremo un boost delle performance di 4 volte.

Guasti

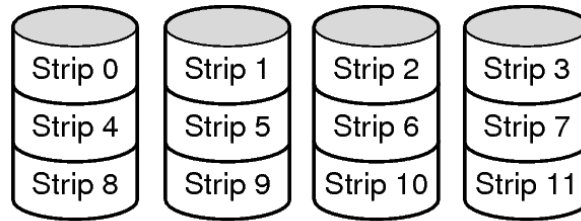
Un guasto su un disco qualsiasi, si propaga sul volume logico RAID: aumentando la ridondanza dei dati, migliora l'affidabilità. Esistono inoltre meccanismi di sostituzione dei dischi tramite dischi di riserva (prima dei guasti non fanno parte del sistema RAID), che vengono ripopolati il prima possibile redistribuendo i dati.

7.12 RAID

Redundant Arrays of Inexpensive Disks, noti anche come Redundant Arrays of Independent Disks. Esistono vari livelli dei sistemi RAID.

7.12.1 RAID 0 - *striping*

Effettua **striping** in modalità **round-robin**. Ottiene prestazioni ottimali tramite un approccio semplice e con letture di grandi volumi.

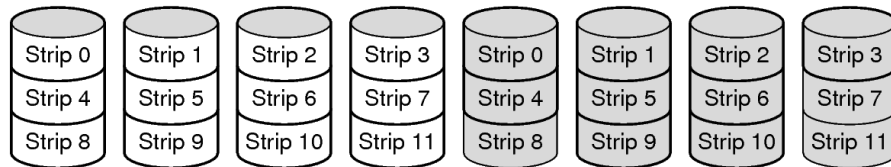


Nella migliore delle ipotesi, con n richieste ed m dischi, ci saranno

$$\frac{n}{m} \text{ richieste/disco}$$

*E se sono sfortunato :(? Tutte le richieste saranno sullo stesso disco. Fortunatamente, nei grandi numeri, la distribuzione delle richieste è più vicina al best case che al worst. Inoltre, **non usa ridondanza**: sono quindi **molto vulnerabili ai guasti**, considerando che ogni disco può guastarsi. Teoricamente, anche se i dischi di oggi sono molto sicuri, la probabilità che si guastino aumenta col numero di dischi del sistema RAID.*

7.12.2 RAID 1 - *mirroring*

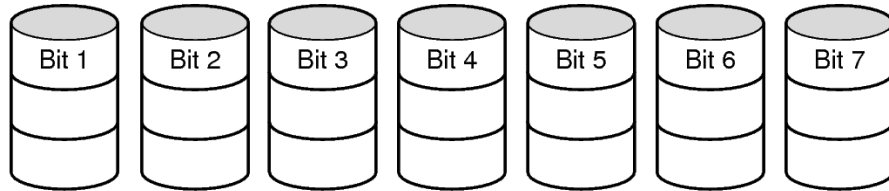


Gli eventuali stripe sono anche chiamati **duplicati**, ma può anche essere usato senza striping. Le prestazioni in lettura raddoppiano (in tutti i casi), ma a causa del doveroso aggiornamento di ambo i dischi, non abbiamo aumenti di performance per la scrittura (rispetto ai RAID 0, se abbiamo striping). Otteniamo tuttavia maggiore tolleranza ai guasti. Caso peggiore in lettura $2\times$, caso peggiore in scrittura $1\times$, caso migliore in scrittura $4\times$. C'è un alto overhead causato dal disco praticamente duplicato. Con n dischi, ne sfrutto $n/2$. *Pago quattro tera per averne due...*

7.12.3 RAID 2 - *striping a livello di bit con ECC*

Lavora sulle word applicando un codice di correzione degli errori ECC. Può correggere singoli bit di errore tramite i **codici di hamming**.

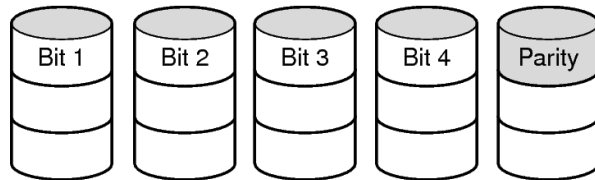
Un esempio? 4 bit con 3 bit di ridondanza. Distribuendo i 7 bit su 7 dischi differenti, e **gestendo opportunamente la sincronia delle rotazioni dei dischi**, si ottiene una forte **fault tolerance**, resistenza ai guasti.



Guastato un disco, se ne fa un altro. Tramite gli altri 6 dischi, è possibile ricostruire l'informazione nel disco perso, qualsiasi esso sia. Con 7 dischi, ne sfrutto 4. L'ECC è overkill.

7.12.4 RAID 3 - *striping a livello di bit con bit di parità*

Aggiunge un solo disco con raccolti i bit di parità.

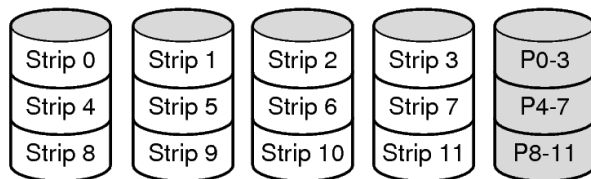


Guastato un disco, fortunatamente, è facile stabilire il valore dei bit persi basandosi sul valore di parità. Offre gli stessi benefici del RAID 2 a costo minore. Serve ancora sincronia tra i dischi. Con 5 dischi, ne sfrutto 4.

I prossimi due RAID sono quelli effettivamente usati.

7.12.5 RAID 4 - *striping a livello di bit con XOR sull'ultimo disco*

È basato sullo **striping a blocchi**. L'ultimo disco contiene solo ridondanza: lo *XOR* dei bit degli strip. Il disco dedicato alla parità è molto utilizzato.



$$P_{0-3} = s_0 \oplus s_1 \oplus s_2 \oplus s_3$$

Guasto di s_2 ?

$s_2 = P_{0-3} \oplus s_0 \oplus s_1 \oplus s_3$ Qualsiasi guasto può essere corretto, anche nei dischi di parità.

- Prestazioni in lettura: $n \times$.
- In scrittura, apparentemente, pessimo costo: $n - 2$ letture e 2 scritture per ogni bit.

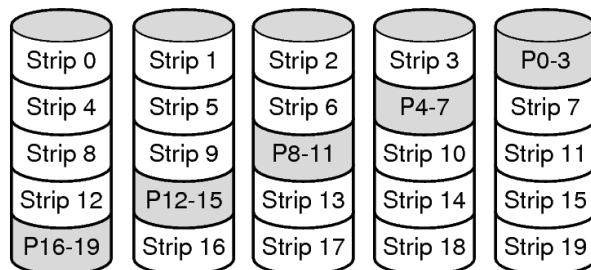
Fortunatamente lo *XOR* può essere ricalcolato anche solo conoscendo il vecchio valore del bit riscritto $s_x \rightarrow \overline{s_x}$.

$$\overline{P_{0-3}} = P_{0-3} \oplus s_0 \oplus \overline{s_0}$$

Questo perché due *XOR* si annullano.

7.12.6 RAID 5 - *striping a livello di bit con informazioni di parità distribuite*)

Risolve il problema relativo all'uso maggiore del quinto disco. Ogni scrittura su un disco, in RAID 4, richiede una scrittura fissa sul quinto disco. Distribuendo le informazioni di parità, distribuiamo il carico di lavoro, con analoga resistenza ai guasti e performance. RAID 5 risolve l'unica problematica di RAID 4.



Un'ultimo vantaggio? Potenzialmente il lavoro è distribuibile su tutti i dischi. Nessun disco ha uno scopo fisso, tutti possono soddisfare richieste, tutti hanno delle informazioni di parità.

7.12.7 Recap sui vari RAID

RAID	Tipo di stripe	Ridondanza	In breve
0	Blocco	No	Sensibile a guasti di qualsiasi disco, velocizza in maniera importante le letture di grandi volumi, soprattutto se questi sono ben distribuiti su più dischi.
1	Blocco	Mirroring (non corregge)	Performance di lettura $\times n$ per n dischi, buona fault tolerance ma alto overhead. Il numero di scritture aumenta per ogni mirror, ma la velocità è limitata dal disco più lento.
2	Bit	Hamming (ECC)	Richiede sincronia a causa dei bit, bottleneck del disco fisico di velocità minima, ottima fault tolerance. Numero di dischi stabilito secondo le regole dell'ECC (2^n). Lo striping a livello di bit è molto pesante.
3	Bit	Parità	Un solo disco per la correzione, stessa fault tolerance di RAID 2 a basso costo. Lo striping a livello di bit è molto pesante.
4	Blocco	XOR	Un singolo disco conterrà gli XOR di ogni blocco. Non necessita sincronia, ottima fault tolerance. L'aggiornamento dei blocchi di parità è ottimizzato calcolando lo XOR dei vecchi valori di parità con i nuovi valori dei blocchi aggiornati.
5	Blocco	XOR distribuito	Distribuendo i blocchi di parità su tutti i dischi fisici, si crea meno bottleneck, si distribuisce l'usura su tutti i dischi del volume RAID. Ogni disco ha pari responsabilità.

7.13 Solid State Disk - SSD

Sono dispositivi basati su memorie flash. Usano tecnologie **NAND**, con architetture più simili alle RAM, ma dalla natura non volatile. Offrono **letture molto veloci**, grazie all'**assenza di parti meccaniche**.

Per questo motivo, gli SSD ignorano i meccanismi di scheduling del disco, questi vengono inibiti, in quanto innessari: basta un approccio FIFO per ottenere prestazioni ottimali.

7.13.1 Prestazioni

Gli SSD si usurano **ad ogni operazione di cancellazione**. Ogni riscrittura di (parte di) un blocco avviene previa cancellazione dell'intero blocco. Definiamo blocco come un'unità di cancellazione. Un blocco è composto da pagine, definite come **unità di allocazione**.

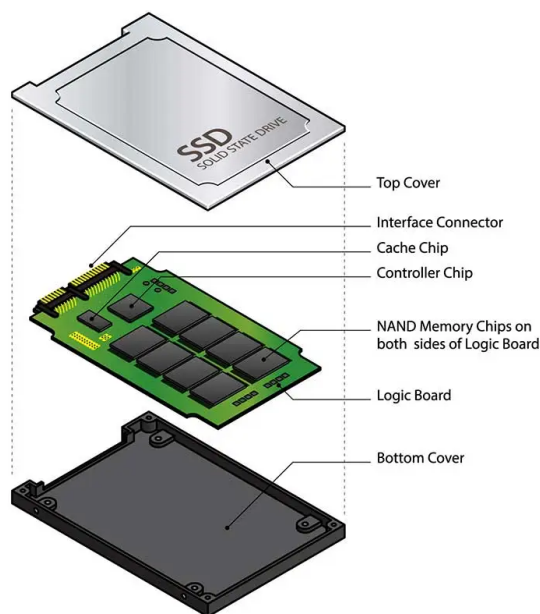
Le letture sono effettuate su pagine, le cancellazioni sono sugli interi blocchi. Per aggiornare un'informazione di una pagina, va effettuata la cancellazione e riscrittura dell'intero blocco.

7.13.2 Memorie Flash e File System

Cancellare ad oltranza singole pagine, usura rapidamente blocchi. Bisogna avere un file system ottimale per minimizzare il deterioramento. Immaginiamo un sistema FAT su un SSD: i blocchi contenenti la FAT verrebbero consumati molto rapidamente.

- **Flash-Friendly File System.**
- **Rimappaggio dei blocchi** tramite controller hardware, detti **Flash Translation Layer**.
A prescindere dal file system, il controller gestirà una sorta di mappa (stile tabella delle pagine) che si pone tra memoria effettiva e file system, mappando blocchi logici a blocchi effettivi. Il controller, in fase di riscrittura di un blocco, invece di cancellarlo, rimapperà lo stesso blocco logico ad un nuovo blocco fisico, in modo da uniformare l'usura dei blocchi.

Una sovrascrittura di un blocco logico, coinciderà con una scrittura ad un altro blocco fisico.



7.13.3 Garbage Collection e TRIM

Le celle piene di **spazzatura** (file modificati e cancellati), vengono cancellate solo quando è necessario. La cancellazione all'ultimo momento possibile, permette di scegliere il miglior blocco da cancellare: un blocco con tante pagine da eliminare e poche da mantenere, sarà migliore.

Il controller stesso, crea spazzatura: i blocchi cancellati nel volume logico, rimangono nel sistema fisico. Il controller non ha visibilità relativa a quali file sono spazzatura o meno: in fase di riscrittura, cancellerà anche i file non più necessari all'OS, usurando velocemente e rendendo lenti dischi virtualmente vuoti. L'operazione **TRIM**, offerta dal Controller al Sistema Operativo, permette di segnalare quali blocchi possono essere ignorati in fase di cancellazione. Il Sistema Operativo potrà informare il controller in fase di cancellazione.